

D5: Finite Automata and Turing Machines



NO annotations
this week due to
technical difficulties
during discussion in

Sec 17: Fr 11:30am-12:30pm FXB 1024
IA: Pearl Lin

Agenda

- ▶ Admin
- ▶ Computability intro
- ▶ Deterministic finite automata (DFAs)
- ▶ Turing machines
- ▶ Worksheet

Admin

- ▶ HW 5 is out, due Wednesday 10/7

Computability Terminology

Alphabets and Strings

Alphabet: a nonempty, finite set of symbols (denoted Σ)

- ▶ $\Sigma_{binary}: \{0,1\}$ consists of only symbols 0 and 1
- ▶ $\Sigma_{lowercase}: \{a,b, \dots, z\}$ consist of lowercase English letters

String (over an alphabet Σ): a finite, ordered sequence of symbols from Σ

- ▶ Any binary sequence is a string over Σ_{binary}
- ▶ Any lowercase English word is a string over $\Sigma_{lowercase}$
- ▶ Empty string is possible: denoted by ϵ
- ▶ **Set power notation** is used to define all strings of a certain length
 - ▶ Σ^* denotes the set of all strings over Σ
 - ▶ $\{0,1\}^2 = \Sigma_{binary}^2 = \{00,01,10,11\}$ (all binary strings of length 2)
 - ▶ $\{0,1\}^* = \Sigma_{binary}^* = \{\epsilon, 0,1,00,01,10,11,000,001, \dots\}$ (all binary strings)

Important: Σ^* may be an infinite set, but each string inside Σ^* is still of finite length.

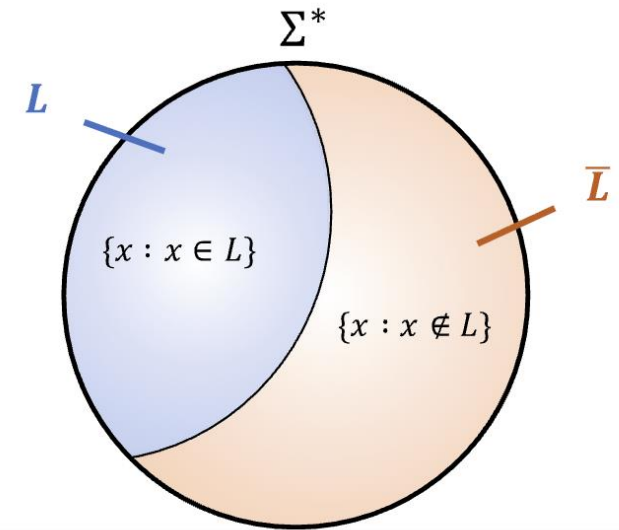
Decision Problems and Languages

Decision problem: a problem that has a yes/no answer

- ▶ Is there a flight from Seattle to Detroit for less than 200 dollars?
- ▶ Can be generalized to take in input (a string):
 - ▶ For input $s = (x, y, z)$: "Is there a flight from x to y for less than z dollars?"

Language (of a decision problem): set of "yes" inputs to the decision problem, a subset of Σ^*

- ▶ $L_{FLIGHTS} = \{(x, y, z) : \text{there is a flight from } x \text{ to } y \text{ for less than } z \text{ dollars}\}$
- ▶ "Is there a flight from x to y for less than z dollars?" is **equivalent** to checking if $(x, y, z) \in L_{FLIGHTS}$
- ▶ Since a language is a set, all set operations are valid
 - ▶ $\overline{L_{FLIGHTS}} = \{(x, y, z) : \text{there is no flight from } x \text{ to } y \text{ for less than } z \text{ dollars}\}$



Wait! What About Other Inputs?

- ▶ Can we have graphs, arrays, or even other algorithms (spoiler) as inputs to an algorithm?
- ▶ Yes! As long as they are encoded into a string
 - ▶ Use angle brackets $\langle \rangle$ to denote an encoding
 - ▶ Ex. $\langle G \rangle$ is the string encoding of a graph G
 - ▶ Encoding must be **unique** (object can be identified from its encoding)
 - ▶ Encoding must be a **finite-length string**
- ▶ Examples:
 - ▶ Encode an array by listing out its values separated by commas within square brackets
 - ▶ $\langle A \rangle = [1,2,3,4]$
 - ▶ Encode an algorithm by filling a string with its C++ source code

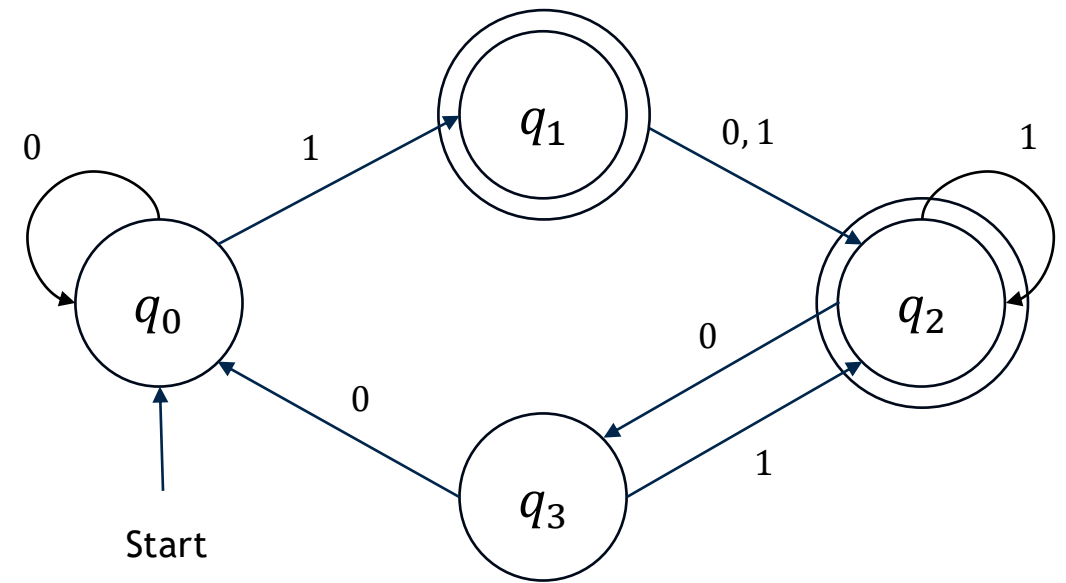
Some String Regex/Notation

- ▶ **Parenthesis ()**: treat whatever is inside as a **single clause**
 - ▶ "ababab" matches $(ab)^3$
 - ▶ "aba", "aaabbb", and "ababba" do NOT match $(ab)^3$
- ▶ **Kleene star ***: look for **zero or more** of the clause to the left
 - ▶ ϵ , "ab", "abab", "ababab", etc. match $(ab)^*$
- ▶ **Power k** : look for **exactly k power** of the clause to the left
 - ▶ Only "jjjj" matches j^4
- ▶ **Bar |**: look for **either** the clause to the right or the clause to the left
 - ▶ (CSE | ECE) matched either "CSE" or "ECE"
- ▶ **Plus +**: look for **1 or more** of the clause to the left
 - ▶ "ab", "abab", "ababab", etc. match $(ab)^+$

Deterministic Finite Automata (DFA)

DFA

- ▶ Defined as a 5-tuple $(Q, \Sigma, q_0, F, \delta)$
 - ▶ Q : finite set of all DFA states $\{q_0, q_1, q_2, q_3\}$
 - ▶ Σ : input alphabet that the DFA can read $\{0,1\}$
 - ▶ $q_0 \in Q$: initial (starting) state q_0
 - ▶ $F \subseteq Q$: set of DFA “accept” states $\{q_1, q_2\}$
 - ▶ δ : transition function $Q \times \Sigma \rightarrow Q$ determining which state to go to next based off current state and input symbol
 - ▶ Can be represented by a table
 - ▶ Value of cell at (row, column) is the next state for each row (current state) and column (input symbol)



State\Input	0	1
q_0	q_0	q_1
q_1	q_2	q_2
q_2	q_3	q_2
q_3	q_0	q_2

DFA (continued)

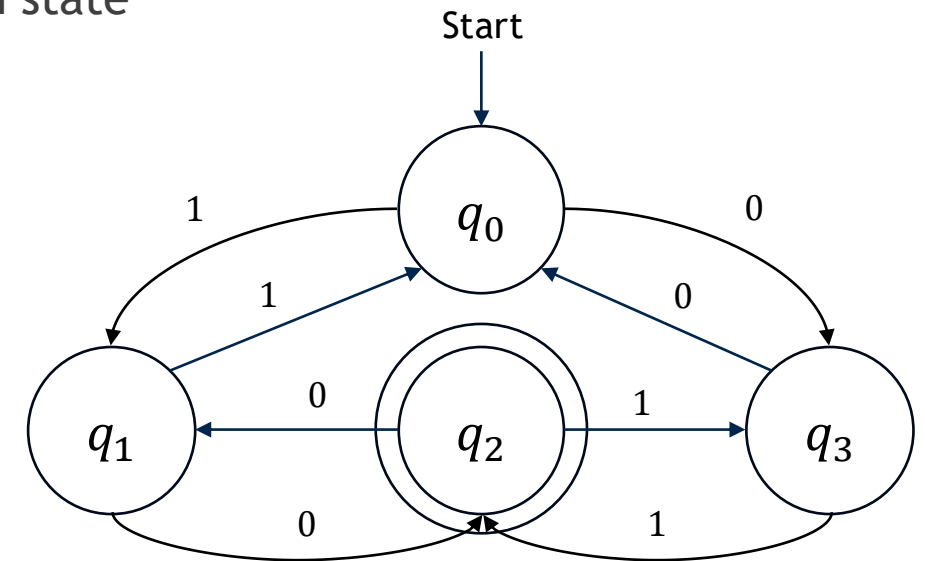
How DFA's work:

1. Start at initial state q_0
2. Read input **1 symbol at a time exactly once**, using transition function δ to determine where to go
3. Once the input string has been parsed entirely, examine end state
 - ▶ If end state is an accept state, DFA accepts
 - ▶ Else end state is a reject state, DFA rejects

Language (of a DFA M): set of inputs that M accepts
 $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$

Decides (for a DFA M): A DFA M decides a language L if

1. M correctly accepts for all $x \in L$
2. M correctly rejects for all $x \notin L$



DFA decides
 $L = \{x \in \{0,1\}^* : x \text{ contains an odd number of 0's and 1's}\}$

Languages that DFA Can't Decide?

- ▶ Language $L = \{0^n 1^n : n \in \mathbb{N}\}$
- ▶ **Fact: No DFA can decide this language!** Proof idea:
 - ▶ For any arbitrary DFA, consider an input whose size exceeds the # of states s in the DFA (i.e., $n = s + 1$)
 - ▶ By Pigeonhole Principle, there must be some state that is repeated twice while input is being parsed
 - ▶ Find 2 distinct strings $s_1 \in L$ and $s_2 \notin L$ that arrive at the same state at some point while being parsed
 - ▶ Show that the DFA will produce the same output for s_1 and s_2 despite only one of them being in L
- ▶ Languages decidable by DFA's are called **regular languages**

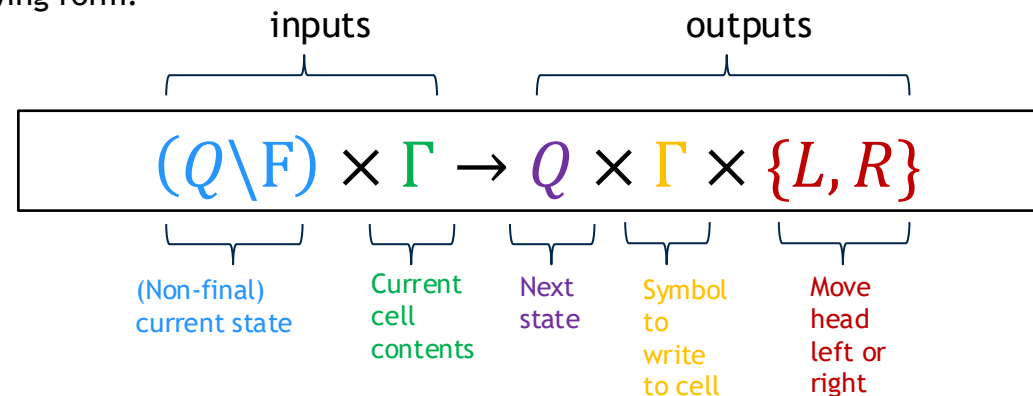
More Complex DFA's

- ▶ **Fact: the intersection and union of regular languages is also a regular language**
- ▶ Given a DFA M_1 that decides a regular language L_1 and a DFA M_2 that decides a regular language L_2 , it's possible to construct a DFA M_3 that decides $L_1 \cup L_2$ and $L_1 \cap L_2$ using **product construction**
 - ▶ Not required for this class, but can be helpful

Turing Machines

TM

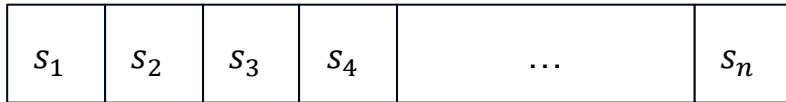
- ▶ Defined as a 7-tuple $(\Sigma, \Gamma, Q, q_{start}, q_{acc}, q_{rej}, \delta)$
 - ▶ Σ : input alphabet that the TM can read
 - ▶ Γ : tape alphabet containing symbols that can be written to cells on the tape
 - ▶ Must have Σ as a subset ($\Sigma \subseteq \Gamma$)
 - ▶ Must have a special blank symbol that is in the tape alphabet but not the input alphabet ($\perp \in \Gamma$ but $\perp \notin \Sigma$)
 - ▶ Q : finite set of all TM states
 - ▶ $q_{start} \in Q$: initial (starting) state
 - ▶ $q_{acc}, q_{rej} \in Q$: accept and reject states
 - ▶ Must be distinct ($q_{acc} \neq q_{rej}$)
 - ▶ $F = \{q_{acc}, q_{rej}\}$ are the final states: if the machine ever transitions to either of these states, computation immediately **halts**
 - ▶ δ : transition function of the following form:



DFA's vs Turing Machines

DFA

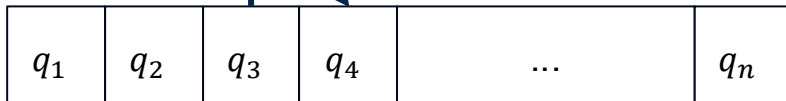
Finite tape



Transition Function δ

Input: current state,
current tape symbol

Output: next state



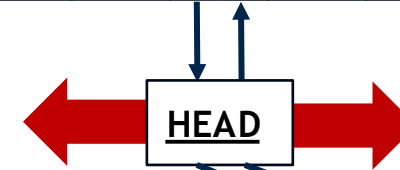
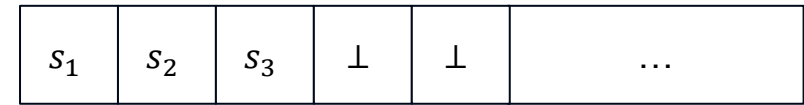
Finite sequence of states

Possible results:

- Accept
- Reject

Halts when end of
finite tape is reached

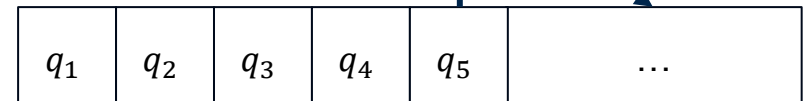
Infinite tape



Transition Function δ

Input: current state,
current tape symbol

Output: next state, new
tape symbol, head goes
left/right



Sequence of states

- Infinite if TM loops
- Finite if TM halts

Halts when q_{acc} or
 q_{rej} is reached
(could be never!)

TM

Church-Turing Thesis

A function can be computed by an algorithm if and only if it can be computed by a Turing Machine

- ▶ Any algorithm (even the ones you write with C++, Python, MATLAB, etc.) can be modeled by a Turing machine
- ▶ When you're asked to give a Turing Machine, it suffices to use **pseudocode** in place of explicitly defining all the TM components (unless you're asked to otherwise)

Languages, Decidability, Recognizability

Language (of a TM M): set of inputs that M accepts
 $L(M) = \{x \in \Sigma^* : M \text{ accepts } x\}$

- ▶ Anything not in $L(M)$ is an input the TM **rejects** or **loops** on

Decides (for a TM M): A TM M decides a language L if

1. M correctly accepts for all $x \in L$
2. M correctly rejects for all $x \notin L$

- ▶ Conditions for decidability imply the the TM **must halt on all inputs** (no looping)

Decidability: A language L is decidable if there exists some TM that decides it

Recognizes (for a TM M): A TM M recognizes a language L if

1. M correctly accepts for all $x \in L$
2. M correctly loops or rejects for all $x \notin L$

- ▶ In contrast conditions for recognizability imply that the the TM **can loop**

Recognizability: A language L is recognizable if there exists some TM that recognizes it

A TM That Decides $L = \{0^n 1^n : n \in \mathbb{N}\}$

```
decide_L(x) {  
    i, count_0, count_1  $\leftarrow$  0  
    while (i < x.size() && x[i] == '0') {  
        i  $\leftarrow$  i + 1  
        count_0  $\leftarrow$  count_0 + 1  
    }  
    while (i < x.size() && x[i] == '1') {  
        i  $\leftarrow$  i + 1  
        count_1  $\leftarrow$  count_1 + 1  
    }  
    return count_0 == count_1  
}
```

Pseudocode is completely valid! Any algorithm can be modeled by a TM by the Church-Turing thesis.

Implies that TM model is more powerful than DFA model (among other models, see Chomsky Hierarchy)

Worksheet

Q2A

2. For each of the following decision problems, (i) define a reasonable (finite) alphabet Σ , (ii) give the encoding (over the alphabet) of a representative input object, and (iii) define a language L for the corresponding decision problem. Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet.
- (a) Given a binary string b , does it have an even number of 0s?

Q2A (annotated)

2. For each of the following decision problems, (i) define a reasonable (finite) alphabet Σ , (ii) give the encoding (over the alphabet) of a representative input object, and (iii) define a language L for the corresponding decision problem. Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet.
 - (a) Given a binary string b , does it have an even number of 0s?

Q2A (solution)

2. For each of the following decision problems, (i) define a reasonable (finite) alphabet Σ , (ii) give the encoding (over the alphabet) of a representative input object, and (iii) define a language L for the corresponding decision problem. Use the notation $\langle X \rangle$ for the encoding of object X as a string over the alphabet.

(a) Given a binary string b , does it have an even number of 0s?

- (a)
- i. $\Sigma = \{0, 1\}$.
 - ii. For a binary string b , write it as a string of bits. For example, if b is the binary string “010”, then $\langle b \rangle = 010$.
 - iii. $L = \{ \langle b \rangle : b \text{ is a binary string with even number of 0s.} \}$

Q3

3. True or False: there is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}.$

Justify your answer.

Q3 (annotated)

3. True or False: there is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}.$

Justify your answer.

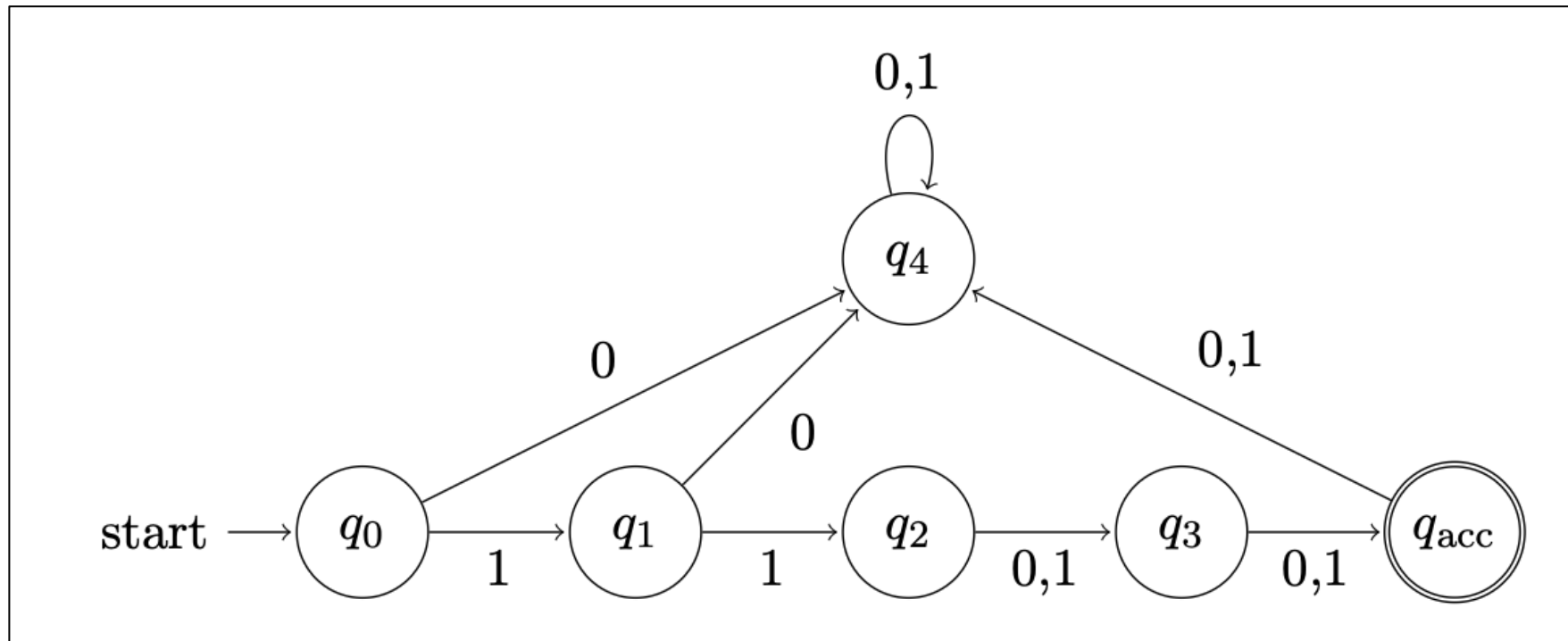
Q3 (solution)

3. True or False: there is a DFA that decides the following language:

$\{x : x \text{ is a binary string of length 4 that starts with two 1s}\}.$

Justify your answer.

Yes, it exists:



Q4

4. True or False: there is a DFA that decides the following language: $\{x : x \text{ is a palindrome}\}$, where the alphabet Σ has at least two elements. Justify your answer.

Q4 (annotated)

4. True or False: there is a DFA that decides the following language: $\{x : x \text{ is a palindrome}\}$, where the alphabet Σ has at least two elements. Justify your answer.

Q4 (solution)

4. True or False: there is a DFA that decides the following language: $\{x : x \text{ is a palindrome}\}$, where the alphabet Σ has at least two elements. Justify your answer.

An arbitrary DFA must have a finite number of states s

Consider a string of k 0's, where $k > s$. By the Pigeonhole principle, then there must be distinct indices $1 \leq i < j \leq k$ such that the state after reading 0^k and the state after reading 0^j are the same

Let $x_1 = 0^i 1 0^i$

Let $x_2 = 0^k 1 0^i$

After reading $0^i/0^k$, the state of the DFA is the same for x_1 and x_2 . From here on outward, the remaining unparsed portion of the string (10^i) is the same between x_1 and x_2 , so the DFA must output the same decision for both. However, at least one these decisions is wrong given that x_1 is a palindrome and x_2 isn't.

Section 2.3 Q1

1. (True/False) A Turing machine can potentially decide more than one distinct language.

Section 2.3 Q1 (annotations)

1. (True/False) A Turing machine can potentially decide more than one distinct language.

Section 2.3 Q1 (solution)

1. (True/False) A Turing machine can potentially decide more than one distinct language.

False.

For languages to be distinct, there must exist some input x that is in one language but not in the other.

A Turing Machine that decides more than 1 distinct language would then have to accept in some instances and reject in others. This is not possible since Turing Machines are deterministic.

Section 2.4 Q1

Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

Section 2.4 Q1 (annotations)

Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

Section 2.4 Q1 (solution)

Let A and B be languages. $A \triangle B$ denotes the *symmetric difference* of the sets A and B . That is, $A \triangle B = (A \cup B) \setminus (A \cap B)$. This can also be seen as the set of strings that are in *exactly one* of A or B .

Suppose that A and B are decidable. Then $A \triangle B$ is decidable for (all/some/no) such A, B . Provide justification to support your answer.

All.

Since A and B are decidable, there exist TM's M_A and M_B that decide A and B respectively.

We can create a new TM $M_{A \triangle B}$ that accepts if exactly one of M_A, M_B accepts, and otherwise rejects. This TM decides the language $A \triangle B$, so indeed it is decidable.

Additional Slides

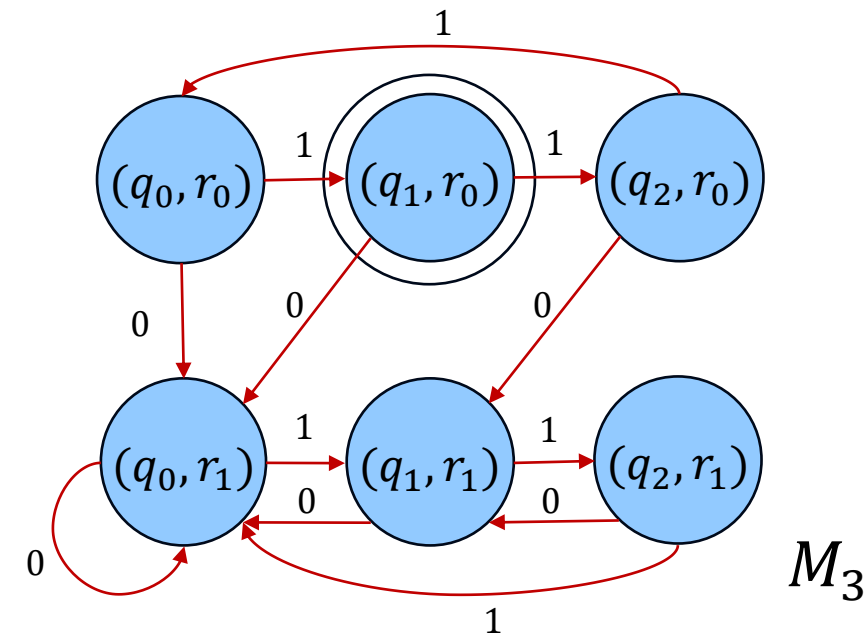
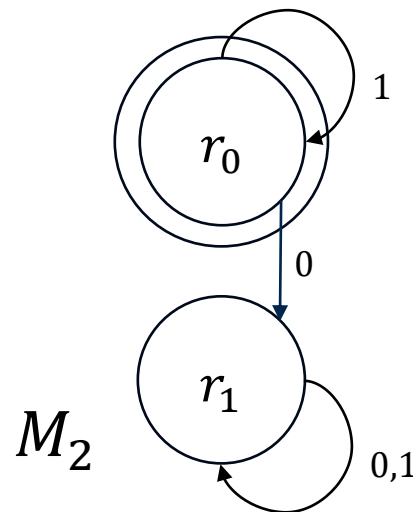
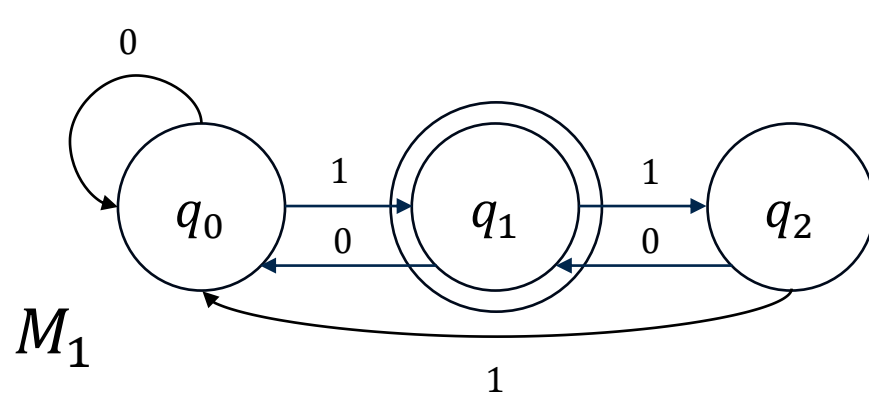
Proof that No DFA Decides $L = \{0^n 1^n : n \in \mathbb{N}\}$

- ▶ Let M be an arbitrary DFA, and s be the number of states of M . We will exhibit an input that M returns an incorrect output for.
- ▶ When run on an input starting with 0^{s+1} , some state $q \in Q$ **must be repeated** while M processes the string. Specifically, there exists $0 \leq i < j \leq s + 1$ such that M is at the same state after processing 0^i and 0^j .
- ▶ Consider strings $s_1 = 0^i 1^i$ and $s_2 = 0^j 1^i$. By inspection we can see that $s_1 \in L$ but $s_2 \notin L$.
- ▶ M outputs the **same decision** for both s_1 and s_2 , because M is at the same state q after parsing 0^i and 0^j . After that, s_1 and s_2 have the same remaining input (1^i).
- ▶ However, only $s_1 \in L$! This means that M **outputs the wrong decision for one of them** and thus does not decide L .

DFA Product Construction

- ▶ Suppose we have a DFA M_1 that decides L_1 and a DFA M_2 that decides L_2
- ▶ Goal: construct a DFA M_3 from M_1 and M_2 that decides $L_1 \cup L_2$ or $L_1 \cap L_2$
- ▶ Idea: run M_1 and M_2 at the same time, and keep track of the states for each
 - ▶ States for M_3 will be **all possible pairs of states** of M_1 and M_2
 - ▶ **Transitions between states** for M_3 should take into account state transitions of M_1 **AND** M_2
 - ▶ Mark accept states in M_3 to match $L_1 \cup L_2$ or $L_1 \cap L_2$

M_3 accept state(s) are marked for $L_1 \cap L_2$ (need both q_1 AND r_0)

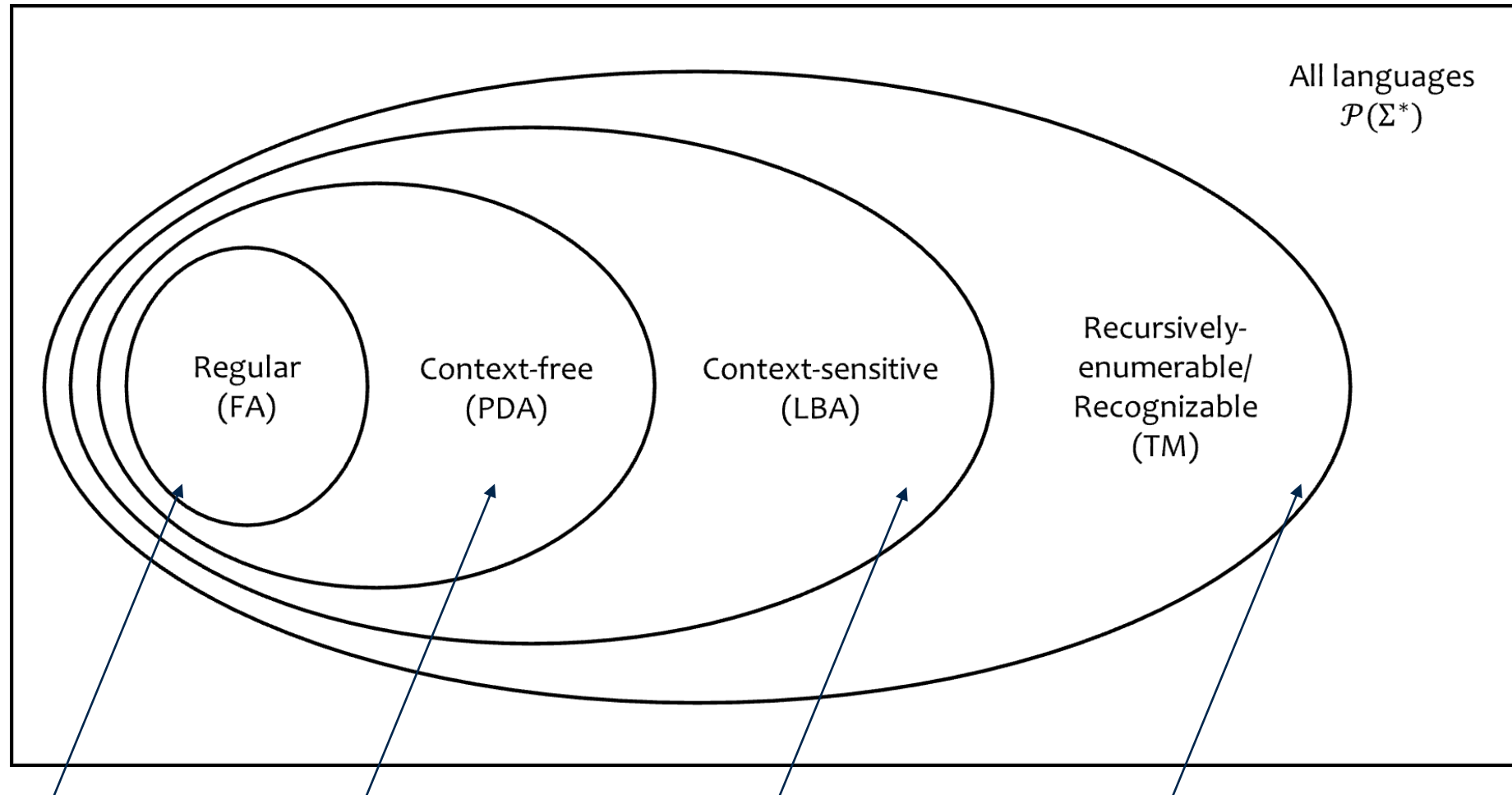


DFA Product Construction (continued)

- ▶ Formal definition:
- ▶ Given 2 DFA's $M = (Q, \Sigma, q_0, F, \delta)$ and $M' = (Q', \Sigma, q'_0, F', \delta')$, define the **product construction** N of M and M' to be the **5-tuple** $(Q, \Sigma, (q_0, q'_0), F_N, \delta_N)$ where:
 - ▶ $Q = Q \times Q'$
 - ▶ $\delta_N : Q \times \Sigma \rightarrow Q$ satisfying $\delta_N((q, q'), c) = (\delta(q, c), \delta'(q', c))$ for all $q \in Q, q' \in Q', c \in \Sigma$
- ▶ Suppose M decides L_1 and M' decides L_2
- ▶ Once you have N , mark the accept states as follows:
 - ▶ For $L_1 \cup L_2$, mark (q, q') as an accept state if $q \in F$ or $q' \in F'$
 - ▶ For $L_1 \cap L_2$, mark (q, q') as an accept state if $q \in F$ and $q' \in F'$

Chomsky Hierarchy

 Memory power



Finite automata (FA):
no memory beyond its
own states

Push down automata (PDA):
single stack for memory,
access only top of stack

Linear bounded automata (LBA):
memory is linear with input size,
access any memory point

Turing Machine (TM):
unbounded memory, access
any memory point